Indian Institute of Technology Jammu

Mid-Semester Examination — Computer Systems and Programming (CS-1-01 MO)

WORKED SOLUTIONS

Time: 1 Hour 30 Minutes	**Maximum Marks: 40**
Name: _____	Roll No.: _____

Instructions:

1. Attempt as many questions as you like — there is no penalty for extra attempts. Your **best 4 of 6** count in every section. The marks per question are printed beside each question.
2. In Sections A and B write only the letter of your choice.
3. **In Section C, show your working** — a correct program counts as its own working. Answers without steps earn partial credit at best.
4. **No electronic devices** — including mobile phones, smartwatches, tablets, laptops and calculators — are permitted.
5. Students must **not communicate with, or exchange materials with, other students** during the examination.
6. Any form of **unfair means or academic misconduct** will be dealt with according to the Institute's examination rules.

SECTION A — Multiple Choice

Best 4 of 6. $4 \times 2 = 8$ Marks

1. `int day[10];` The value for the **7th** day is stored in: **2 pts**

   A. `day[7]` B. `day[8]`
   C. `day[0]` D. `day[6]`

   Answer. D — `day[6]`

2.
```
int x = 5;
switch (x) { case 1:  printf("one"); break; case 2:  printf("two"); break; }
printf("done");
```
   What is printed? **2 pts**

   A. nothing at all B. `onedone`
   C. `done` D. a compile error

   Answer. C — `done`

3. What does this print?

```
printf("%d", (5 > 3) + (2 > 4));
```
   2 pts

   A. 0 B. 2
   C. `true` D. 1

   Answer. D — 1

4.
```
for (int i = 0; i < 6; i++) {
  if (i % 3 == 0) continue;
  printf("%d ", i);
}
```
   What is printed? **2 pts**

   A. `0 1 2 4 5` B. `1 2 4 5`
   C. `1 2 3 4 5` D. `0 3`

   Answer. B — `1 2 4 5`

5. `int x = 4;` then `x *= 2 + 3;` What is `x`? **2 pts**

A. 20 B. 11
C. 9 D. 5

Answer. A — 20

6. In 4-bit **two's complement**, which pattern represents -6? **2 pts**

A. `1010` B. `1001`
C. `1110` D. `0110`

Answer. A — `1010`

SECTION B — Multiple Choice

Best 4 of 6. $4 \times 3 = 12$ Marks

1. All operands are `int`. What is the value of `-7 / 2 * 2 + 7 % 2`? **3 pts**

A. -1 B. -5
C. -7 D. -6

Answer. B — -5

2.
```
int age = 8;
if (age < 60) printf("full");
else if (age < 12) printf("child");
else printf("senior");
```

What is printed? **3 pts**

A. `full` B. `child`
C. `senior` D. nothing is printed

Answer. A — `full`

3. On a 4-bit two's-complement machine, which of these additions does **not** overflow? **3 pts**

A. $5 + 4$ B. $(-7) + (-2)$
C. $(-8) + (-3)$ D. $3 + 4$

Answer. D — $3 + 4$

4.
```
int a = 64, b = 40, t;
while (b != 0) { t = a % b; a = b; b = t; }
printf("%d", a);
```

What is the output? **3 pts**

A. 0 B. 8
C. 24 D. 16

Answer. B — 8

5. `day[i]` is a day of the month; `level[i]` is the level recorded on that day.

```
int day[8]   = {  9,  4, 17,  2, 23, 11,  6, 14 };
int level[8] = { 52, 17, 38, 61, 29, 45, 33, 47 };
```

What level was recorded on day 17? **3 pts**

A. 17 B. 61
C. 38 D. 29

Answer. C — 38

6.
```
int i = 0;
while (i < 4) {
  switch (i) { case 2:  printf("q"); break; default:  printf("%d", i); }
  i++;
}
```
What is printed? **3 pts**

A. `01q` B. `0123`
C. `01q3` D. `01q23`

Answer. **C** — `01q3`

SECTION C

Best 4 of 6. $4 \times 5 = 20$ Marks

1. A student classifies a triangle from three sides `a`, `b`, `c` like this: **5 pts**

```
if      (a==b && b==c)              printf("equilateral");
else if (a==b || b==c || a==c)      printf("isosceles");
else                                printf("scalene");
```

(a) Give three side lengths for which this prints a confident but **wrong** answer, and say what it prints. **2 pts**

(b) Explain in one sentence what is missing. **1 pts**

(c) Write the corrected structure. **2 pts**

Answer. (a) e.g. $2, 3, 9$ — it prints `scalene`. (b) It never checks that the three lengths form a triangle at all. (c) Test validity *before* classifying.

Working. **(a)** With $a = 2$, $b = 3$, $c = 9$: no two sides are equal, so the ladder falls to the last rung and prints `scalene`. But $2 + 3 = 5$, which is less than 9 — those three sticks cannot close into a triangle. The program is confidently wrong, and nothing in it can notice.

(b) Three lengths form a triangle only if *every* pair sums to more than the third. That test is absent, so the ladder classifies a shape that does not exist.

(c)

```
if      (a+b <= c || b+c <= a || a+c <= b)  printf("invalid");
else if (a==b && b==c)                      printf("equilateral");
else if (a==b || b==c || a==c)              printf("isosceles");
else                                        printf("scalene");
```

The order is the whole answer: validity is a *precondition*, so it must be the first rung.

2. Write a complete C program that reads one integer `n` between 1 and 99999. While `n` has more than one digit, replace `n` by the **sum of its digits**. Print the single digit that remains, in the form `root=<D>`.

For example 9875 becomes 29, then 11, then 2, so it prints `root=2`.

Marks are for correctness, not for elegance. **5 pts**

Answer. Two loops, one inside the other: the inner one adds up the digits of `n`, the outer one repeats that while `n` still has more than one digit.

Working.

```
#include <stdio.h>
int main(void) {
    int n;
    scanf("%d", &n);
    while (n > 9) {
        int s = 0;
        while (n > 0) { s += n % 10; n /= 10; }
        n = s;
    }
    printf("root=%d\n", n);
    return 0;
}
```

Two ideas carry the marks. `n % 10` is the last digit and `n / 10` throws that digit away, so the inner loop walks the digits from the right and stops when nothing is left. And the outer condition is `n > 9`, not `n != 0`: a single digit is already the answer, so the work stops the moment `n` falls below 10.

pass	digits summed	new `n`
1	$9+8+7+5$	29
2	$2+9$	11
3	$1+1$	2 — single digit, stop

Note two cases worth testing. A single-digit input such as 7 must print `root=7` without entering the outer loop at all. And 99999 sums to 45, then 9 — the outer loop runs exactly *twice*, which is already more than a program written with an `if` instead of a loop can manage.

Accept any correct program. Building the answer with a `for`, or summing into `n` itself with a second variable, is equally right.

3.
```
int k = 1, N = 7;
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        int e = i;
        if (j     < e) e = j;
        if (N-1-i < e) e = N-1-i;
        if (N-1-j < e) e = N-1-j;
        if (e == k) printf("*");
        else        printf(".");
    }
    printf("\n");
}
```

(a) Write out exactly what this prints. **3 pts**

(b) What is the largest value of `k` for which it prints at least one `*`, and what does it print then? **2 pts**

Answer. (a) a 5×5 square of stars, one cell in from the edge, hollow. (b) $k = 3$ — a single `*` at the centre.

Working. The four `if`s that touch `e` leave it holding the *smallest* of i, j, $N-1-i$ and $N-1-j$ — that is, how far cell (i, j) sits from the **nearest** of the four edges. A cell is starred when that distance is exactly `k`, so the picture is the set of cells one step in from the border:

```
.......
```

```
.*****.
.*...*.
.*...*.
.*...*.
.*****.
.......
```

Row 0 and row 6 are blank because their distance is 0, not 1; the corners of the inner square are at $(1,1)$, $(1,5)$, $(5,1)$ and $(5,5)$.

(b) As `k` grows the square shrinks toward the middle. The centre of a 7×7 grid is $(3,3)$, whose distance to the nearest edge is $\min(3,3,3,3) = 3$, so $k = 3$ is the last value that matches anything — and it matches that one cell alone:

```
.......
.......
.......
...*...
.......
.......
.......
```

At $k = 4$ nothing matches: no cell of a 7-wide grid is four steps from every edge. In general the largest useful `k` is $\lfloor (N-1)/2 \rfloor$.

The point: one predicate, changed by a single number, walks the squares from the border inward. That is what it means to describe a shape by a *rule* rather than by counting spaces.

4. All in **4-bit two's complement**, showing every carry. **5 pts**

(a) Compute $6 + 3$. State the decimal value the stored pattern represents, whether overflow occurred, and *how you can tell without converting back*. **3 pts**

(b) Compute $6 - 3$ *as an addition*, using the two's complement of 3. Say what happens to the carry out of the fourth bit. **2 pts**

Answer. (a) `1001`, which is -7; **overflow occurred**. (b) `0011` $= 3$, with the carry out of bit 3 discarded — and *no* overflow.

Working. **(a)** $6 =$ `0110` and $3 =$ `0011`:

bit	6	3	carry in	sum	carry out
3	0	0	1	1	0
2	1	0	1	0	1
1	1	1	0	0	1
0	0	1	0	1	0

The stored pattern is `1001`. Its leading bit is 1, so the value is $9 - 16 = -7$.

Telling without converting back — two tests that always agree. The **sign test**: two positive operands produced a 1 in the sign bit, which no correct sum can do. The **carry test**: the carry *into* the sign bit is 1 but the carry *out* of it is 0, and when those differ the result has overflowed. The true sum is 9, and 4 bits hold only $-8 \ldots 7$.

(b) The two's complement of 3: flip `0011` to `1100`, add 1, giving `1101`. (Check: $2^4 - 3 = 13 =$ `1101`.)

bit	6	-3	carry in	sum	carry out
3	0	1	1	0	1
2	1	1	0	0	1
1	1	0	0	1	0
0	0	1	0	1	0

The carry out of bit 3 is 1. It falls off the 4-bit edge and is discarded, leaving `0011` = 3. And $6 - 3 = 3$. ✓

Put (a) and (b) side by side. In (a) there was **no** carry out and the answer was wrong; in (b) there **was** one and the answer is right. So a carry out of the top bit is not the error flag — overflow is the carry *into* the sign bit disagreeing with the carry *out* of it.

5. Consider this program. **5 pts**

```
int day[8]   = {  2,  5,  9, 12, 16, 21, 25, 28 };
int level[8] = { 41, 38, 52, 47, 52, 33, 52, 45 };
int best = level[0], at = 0;
for (int i = 1; i < 8; i++)
    if (level[i] > best) { best = level[i]; at = i; }
printf("highest = %d, on day %d\n", best, day[at]);
```

(a) What exactly does this print? **2 pts**

(b) Explain why it reports that day, and name the line that decides it. **2 pts**

(c) Change `>` to `>=`. Which day does it report then? **1 pts**

Answer. (a) `highest = 52, on day 9` (b) the **first**, decided by `>` (c) change `>` to `>=`, which reports day 25.

Working. **(a)** The largest value in `level` is 52. Walking the array, `best` becomes 41, then 52 at `i = 2`; nothing later is *strictly* greater, so `at` stays 2 and `day[2]` is **9**.

(b) 52 appears *three* times — at indices 2, 4 and 6, which are days 9, 16 and 25. The comparison `if (level[i] > best)` is strict, so a later equal value does *not* replace the stored one — the code reports the **first** of the three.

(c) Changing `>` to `>=` makes every tie overwrite the record, so the last one wins and it reports day **25**. One character decides which of three equally true answers you get — and the question you were asked decides which character is right.

Why keep `at` at all? Because `best` alone cannot answer the question. The index is what lets you get back to `day[]`.

6.
```
int i = 0, s = 0;
do {
    i++;
    s += i * 2 - 1;
    printf("%d:%d ", i, s);
    if (s > 10 + i) break;
} while (i < 6);
printf("\ni=%d s=%d\n", i, s);
```

(a) Write out exactly what this prints. **3 pts**

(b) Why did the loop stop? **2 pts**

Answer. (a) `1:1 2:4 3:9 4:16` on the first line, then `i=4 s=16`. (b) The `break` ended it — the condition `i < 6` was never false.

Working. **(a)** Two precedence points decide every line. `i * 2 - 1` means $(i \times 2) - 1$, not $i \times (2 - 1)$; and `s > 10 + i` means $s > (10 + i)$, because `+` binds tighter than `>`.

pass	`i`	adds $2i-1$	`s`	break if $s > 10 + i$
1	1	1	1	$1 > 11$? no
2	2	3	4	$4 > 12$? no
3	3	5	9	$9 > 13$? no
4	4	7	16	$16 > 14$? **yes** — break

So the first line is `1:1 2:4 3:9 4:16` (the print happens *before* the test, so `4:16` does appear), and the second is `i=4 s=16`.

(b) Because the loop did not end the way it looks as though it should. The condition `i < 6` is tested at the *bottom* of a `do`–`while`, and it was still true every time it was reached — `i` never got past 4. What stopped the loop was the `break`, which leaves immediately and never consults the condition at all.

Worth noticing: s runs 1, 4, 9, 16. Adding successive odd numbers gives the perfect squares, every time.

— End of Question Paper —